AI Interview Q&A Prep

Forward Deployed AI Engineer · AI Engineer · Generative AI Developer · AI Product Manager

Intermediate to Senior / Lead level

Answers drawn from Scout (Studies Weekly Online) and earlier roles

How to read the levels

Intermediate	Expected of every candidate. Answer clearly, one example.
Senior	Trade-offs, measurements, a real incident.
Lead / Staff	System thinking, roadmap, org-level judgement.

Answer pattern: **Situation -> Task -> Action -> Result (with a number) -> Lesson**

Your 60-second pitch

I'm a senior AI engineer with 10+ years of full-stack work behind me. For the last two years I've been building Scout, the AI assistant inside Studies Weekly Online, a K-12 curriculum platform used by teachers and district admins.

Scout started as a vendor-built LangGraph agent on Amazon Nova Micro behind a buffered API Gateway. I helped take it to what runs today: a custom agent loop on Amazon Bedrock (Claude Sonnet for reasoning, Claude Haiku for fast steps), streaming over a Lambda Function URL, with Pinecone RAG over the curriculum, Bedrock Guardrails tuned for K-12, and a Nuxt 3 frontend where every page tells Scout what it shows and what it can do.

What I'm proudest of is the engineering discipline around it: we declare an evaluation gate before running it, we keep student names away from the model entirely, and when Scout gets something wrong we look first at tool wording, instruction placement and error messages, not at swapping the model.

Before AI, I was a tech lead and lead developer (NestJS/GraphQL, React, Laravel) and I'm a Certified Scrum Product Owner, so I'm comfortable both writing the code and owning the product decision.

Scout cheat sheet

Product	Scout: AI assistant for teachers and district/school admins in Studies Weekly Online (K-12 curriculum).
What it does	Finds curriculum, writes lesson plans/slides/question sets, levels articles to a reading grade, drafts assignments, proposes grades, analyses misconceptions, acts on the page with teacher approval, admin insights.
Frontend	Nuxt 3 / Vue 3 (`frontend-v3/ai/`): chat panel, page surfaces and capabilities, SSE stream parser, executor that verifies actions.
App backend	Laravel 9 / PHP 8.3: auth, rostering, credits as kill switch, action lifecycle with undo, teacher context, memories, admin insights.
AI backend	Python 3.13, FastAPI + streaming Lambda handler, one container image, two doors: API Gateway (threads, levelling) and Lambda Function URL (streamed turn).
Models	Amazon Bedrock: Claude Sonnet (main), Claude Haiku 4.5 (fast steps, guardrail classifier). Earlier: Nova Micro via LangGraph (v1).
Retrieval	Pinecone namespaces `content-v2` (curriculum) and `support` (Help Center). Laravel export -> S3 -> hourly ECS Fargate sync.
State	MongoDB (threads, usage/cost per turn, legacy thread copy); DynamoDB (daily/weekly spend caps, throttles, in-flight leases).
Safety	Bedrock Guardrails (strict + curriculum profile), student pseudonyms in the browser, server-side name scrub, allowlisted facts.
Infra	Terraform (Lambda, API Gateway, Function URL aliases, ECS sync task, alarms, budget), GitLab CI.
Testing	86 frontend pin suites + ~11.5k-assertion verify script; rag `check_*.py` CI checks; Cypress e2e; declared evaluation gates.
Key numbers	v2 baseline 991/991 turns completed; p50 first visible text 3.18 s (target < 2 s, not yet met); mean cost ~$0.017/turn; 7.7% cold-start turns; guardrail false blocks 6 -> 0 of 240 turns after the input-scope fix.

Architecture in one line

Browser (Nuxt `/beta`) -> **Lambda Function URL** (streamed turn) or **API Gateway + authorizer** (threads, levelling) -> one Python container -> **Bedrock** (Claude + Guardrails), **Pinecone**, **MongoDB**, **DynamoDB**; the AI service calls back into **Laravel** with the teacher's own token for context, memories, assignments and approved actions.

1. Opening and behavioural (all four roles)

Q1. Tell me about yourself. [Intermediate]

- Use the 60-second pitch on page 2. End with why this role: you like building AI that real users rely on, end to end, and owning the outcome.

Q2. Walk me through the most significant AI project you've built. [Intermediate]

- **Situation:** teachers needed help finding and adapting curriculum; a first version existed but was slow (buffered), hard to change (LangGraph graph) and did not know what page the teacher was on.
- **Task:** make Scout reliable, fast-feeling, safe for K-12, and able to act inside the app.
- **Action:** streaming turn over a Lambda Function URL; a custom agent loop with ~18 typed tools; page surfaces and capabilities in the Nuxt app; guardrails rescoped to the teacher's words; pseudonyms for students; declared eval gates.
- **Result:** in the final integration run, failed lesson-plan writes went 5/53 -> 2/52, "works well" verdicts 132 -> 147, false guardrail blocks 6 -> 0 of 240, and we removed LangChain/LangGraph entirely with no regression on any chain production passed.

Q3. Tell me about a time you disagreed with a technical decision. [Intermediate]

- Example: a proposal to restructure Scout into "staged prompts" (load details only when needed). I measured what we actually send: a cache point on a 600-token prompt cached nothing, and the chat prefix is ~17.7k tokens that caches well. Staging would break the prompt cache and add round trips.
- I wrote it up with numbers and the recommendation was "don't restructure". Point: disagree with data, write it down, let the owner decide.

Q4. Tell me about a failure. [Intermediate]

- Early on we trusted a test that passed - but it called the function directly, so it passed identically on a build where nothing in the app called that function at all.
- Fix: every new check must be shown to fail first (mutation check); we keep a "measured and failed" register of instrument faults so nobody repeats them.
- Lesson: a green test is a claim; ask what it would look like if it were wrong.

Q5. How do you handle ambiguity in requirements? [Senior]

- Write the smallest design note with a recommended option first and the open questions numbered; get a decision; record it in a decisions log with the date and why.
- Example: the Student Publication Report shows 21.60 (mean of weekly %) vs 20.4 (points / possible). Both are correct; it's a product decision, so I parked it with the product owner rather than "fixing" it in code.

Q6. How do you mentor or raise a team's bar? [Senior]

- Onboarding docs for Scout: a start-here README, local setup, troubleshooting table (symptom -> cause -> check), a decisions log, an offline explorer and an onboarding deck.
- Code review rules that came from incidents ("never truncate silently", "publish answers not rows", "no student name leaves the browser").
- At Groupshop: structured reviews and TDD in React cut production errors ~15%.

Q7. How do you use AI coding tools in your own work? [Senior]

- Daily with Claude Code and Copilot: research, refactors, test pins, reviews. Guardrails: a CLAUDE.md with project invariants, every claim verified against code, gates declared before a run, and a person owns every merge.
- Be honest: AI speeds up the work, but the judgement (what to measure, what to ship) stays with the engineer.

2. AI Engineer / Generative AI Developer - technical

Q8. Explain RAG. How does Scout's retrieval work? [Intermediate]

- RAG = retrieve relevant documents, put them in the prompt, generate a grounded answer. Lowers hallucination and lets content change without retraining.
- Scout: Laravel exports curriculum (articles, questions, resources with grade, publication, unit, week metadata) to S3; an hourly ECS Fargate task (`sync-content`) upserts into MongoDB `content_index` and Pinecone namespace `content-v2`.
- Search is scoped to the teacher's own publications (from Laravel, never from the client), filtered by metadata, then reranked/grouped (several chunks of one source share a title, so we batch by title).
- Safety for the index: a large prune is refused unless forced, with a CloudWatch alarm - a bad export can't wipe the index.

Q9. How do you stop the model from hallucinating? [Intermediate]

- Ground on three sources only (curriculum, the page's own data, Help Center) or decline.
- Tools return answers, not raw rows, and say explicitly when nothing matched: "No material in your publications is linked to <code>" vs "could not check" - never let a failed lookup read as "none".
- Real case: asked about "old classes tucked away", the model said "I found 10 old classes" for an account with none. The fix was the page publishing an explicit `inactive=0` fact and tool wording, not a bigger model.
- Measure it: rubrics that penalise invention and don't punish honest omission.

Q10. What is prompt caching and how did you use it? [Intermediate]

- Bedrock/Claude can cache a stable prefix (system prompt, tool definitions) so later calls pay a fraction for those tokens and get faster first tokens.
- We keep the tool order fixed for caching, and moved per-page data out of the cached prefix so switching pages stops re-paying the cache (ONLINE-7528).
- Measured: cache writes were 63% of the dollars and cache reads ~58% of input tokens; a cache point on a 600-token block caches nothing (below the minimum), so placement matters.

Q11. How do you do streaming from an LLM through AWS? [Intermediate]

- API Gateway REST can't stream and times out at 29 s, so the turn moved to a Lambda Function URL with response streaming; Bedrock `ConverseStream` produces deltas.
- We send typed SSE events: `turn.start`, `status`, `text.delta`, `artifact.start/delta/end`, `pending_action`, `turn.end` / `error`. Every stream ends with exactly one `turn.end` or `error`.
- Browser parser ignores unknown events, so a new event needs both sides - we document that in the troubleshooting table.
- Cancel is explicit (`/turn/cancel` by turn id) because a closed browser doesn't stop a Lambda.

Q12. How do you design tools for an agent? [Senior]

- Small, typed, single-purpose tools with enums the model can actually express (a common failure was an enum that couldn't represent the ask).
- Tool results written for the model: say what happened and what would be right, not just "error".
- Writes never happen in the model: the service proposes a `pending_action`, the teacher approves, the browser executes and then **verifies** by re-reading app state; server actions are logged with undo.
- Real bug: the model passed "1" from the title "Article 1: ..." as an article id. Fix: ids come from the page/tool results, validated, not free text.

Q13. Why did you remove LangChain/LangGraph? [Senior]

- Our flow is a single tool-calling loop; the framework added layers, a checkpoint store and hard-to-debug state for little benefit.
- Replaced with a plain loop over Bedrock `ConverseStream` (max steps, parallel tools per step, turn ceiling).
- Migration risk handled: old threads copied once into `core_legacy_threads` with an `--apply` / `--verify` CLI, run before and after deploy; checkpoints kept until verify is clean in prod.
- Result: no chain production passed failed on the branch; TTFT ratio 1.197x against a 1.2x bar.
- Interview framing: frameworks are great for prototyping; in production, choose what you can debug at 2 a.m.

Q14. How do you evaluate an LLM feature? [Senior]

- Declare the gate **before** any row exists: population, n, pass line, and what a control arm must show. Commit gate, rows and RESULT together.
- Always run a control: one change "passed" 6/6 but the control also passed 6/6 - that's not evidence.
- Use real data, not fixtures - local "graded" answers were junk (1,010 of 1,034 scored full marks; "hdfhfghdfhfh" scored 1.00).
- Mix deterministic checks with an LLM judge, and know the judge's noise: a gate drop turned out to be the judge moving on borderline multiple-choice items.
- Statistics: McNemar for paired pass/fail (leveller movement arm, p = 1.0 -> closed, not shipped); state what n can detect (n=1 detects only a net loss of ~5 chains).

Q15. How do you choose a model? [Senior]

- Quality, latency (TTFT), cost and safety behaviour - measured on our own asks.
- Haiku 5.5 as fast model: better quality (154 vs 147 works-well) but TTFT 1.21x vs 1.2x bar -> held. And as guardrail classifier it labelled 105 of 279 curriculum requests as OTHER -> classifier stays pinned on Haiku 4.5.
- Earlier: Nova Micro was cheap but needed lexical routing and context trimming to stay on-instruction in multi-turn chat; GPT-4o, Gemini and DeepSeek R1 were evaluated in that phase.

Q16. Latency: how did you attack time to first token? [Senior]

- Measure stages: auth callback, context build, first model call, first visible text; separate cold starts (7.7% of turns).
- v2 baseline p50 3.18 s first visible text (p95 6.91 s) vs a < 2 s goal - not met yet, and I say so.
- Levers: prompt caching, fast model with thinking off, show `status` events early, keep per-turn context small ("publish answers, not rows").
- Trade-off kept on file: an output screen for leaks cost ~0.9 s of first-visible time for 23 of 7,104 replies.

Q17. Explain guardrails in your system. [Senior]

- Bedrock Guardrails: a strict profile always, plus a curriculum profile for curriculum-labelled requests (enabled by Terraform flag); ids and versions pinned and refused if unpinned.
- Problem: history curriculum (wars, slavery) tripped content filters, so ordinary teacher requests were blocked and shown as errors.
- Fix: input check reads only the teacher's typed text (`guardContent`), not retrieved articles; output always checked; classifier on Haiku 4.5. Content-check blocks 6 -> 0 of 240 turns.
- Bug fixed: PII ANONYMIZE was treated as BLOCKED and failed the turn; now only a policy that actually fired BLOCKED stops it.
- Caveat I'd mention: dev and prod publish different guardrail versions, so a dev pass doesn't prove prod.

Q18. How do you protect PII, especially children's data? [Senior]

- Invariant: no student name, username or credential reaches the model. Pages publish pseudonyms ("Student 3") from a stable alias module; real names restored only at render.
- `publicFacts()` is an allowlist, not a filter - roster records carry passwords.
- Backstop: server-side name scrub; thread titles never name a person; fail closed (no alias key -> "a student" and the turn waits).
- Infra: rotate exposed secrets, separate prod credentials, MongoDB IAM auth instead of password URIs, security scan fixes reviewed.

Q19. How do you make a page-aware assistant without DOM scraping? [Lead / Staff]

- v1 read the DOM. It was brittle and leaked whatever was on screen.
- v2: components publish what they already know via `useAiSurface()` - subject, view, `content()` and typed capabilities. Surfaces stack by mount depth, so an open dialog shadows the page and disappears when it closes.
- Capabilities call the app's own stores, never DOM elements; the executor runs, then verifies. `forbidden` capabilities can be pointed at but not done.
- Rule: the cost of a feature is what it adds to every turn - check the resting card on each page.

Q20. How do you control cost in production? [Lead / Staff]

- Per-turn cost recorded (`tokenz`); mean ~$0.017/turn.
- Spend caps in code ($5/day, $2.50/week, higher for staff), request throttles per user, in-flight lease -> 429 `too_many_turns`.
- AWS budget alarm, anomaly monitoring, CloudWatch dashboards.
- Honest gap found in audit: per-user limits fail open on a DynamoDB error - a conscious availability-over-cost trade-off, documented.

3. Forward Deployed AI Engineer

Q21. What does a forward deployed engineer do, in your words? [Intermediate]

- Sits with the customer, turns a messy real workflow into a working AI system on their data and infrastructure, and feeds what was learned back to the product team.
- My Scout work was very close to this: I worked with the curriculum team, the product owner and QA on real teacher asks, and shipped into an existing Laravel + Nuxt platform rather than a greenfield app.

Q22. How do you integrate an AI service into an existing, large codebase? [Intermediate]

- Respect the boundary: Scout is a self-contained module (`frontend-v3/ai/`) - nothing outside imports anything but `ai/index.ts`.
- Auth reuse: the browser sends the existing Laravel JWT; the AI service checks it back against Laravel (`api/ai/user/credits`) and gets the user's publication scope from there - one source of truth for access.
- Feature flags per district/school, fail closed; non-AI impact audits so AI branches never change behaviour for users with Scout off.

Q23. A customer says "the AI is wrong". How do you debug? [Senior]

- Get the turn id (every turn has one), pull the CloudWatch log and the thread; reproduce on the same page and data.
- Classify: retrieval (content not exported/synced), context (page didn't publish the fact), tool wording, guardrail, or model.
- Fix the cheapest layer first. In Scout's history the fix was almost never a better model - it was tool result wording, instruction placement, an enum that couldn't express the ask, or an error that said "wrong" without saying what would be right.
- Re-test against a declared pass line with a control, then add a regression pin.

Q24. How do you handle environments and deployments safely? [Senior]

- Terraform per env (`dev.tfvars`, `prod.tfvars`), container image tags, Lambda aliases for streaming rollback.
- Found in a readiness audit: a missing model grant, wrong guardrail profile or bad Mongo/Pinecone secret all passed deploy as green, and `/health` returned healthy with no config at all. So: read startup logs, add smoke turns after deploy, don't trust health checks alone.
- Ordering matters: data migrations run before and after the release; deploy the AI backend before or with the frontend.

Q25. How do you work with non-technical stakeholders (teachers, content team, leadership)? [Senior]

- Write questions they can answer by id ("answer CQ-LV-02 in the box"); give a recommended option first.
- Show real outputs, not metrics alone; keep a decisions log so rulings survive people changing.
- Example: the content team's guardrail topic list was cross-walked to Bedrock topic policies, and the product owner's ruling that wars and "historic barbarians" are real curriculum shaped the curriculum profile.

Q26. The customer's data is messy. What do you do? [Senior]

- Measure it first. 27% of live Spanish weeks had three or fewer substantive lines - thin content explained weak Spanish answers, not the model.
- 57,175 of 57,503 classes still had default calendar dates, so the planning calendar couldn't trust them.
- Publish the gap honestly to the user ("not linked" vs "could not check") and give the content team a concrete list.

Q27. You have two weeks to deliver a pilot at a new customer. Plan? [Lead / Staff]

- Days 1-2: sit with users, collect 30-50 real asks, define 3 success metrics and a pass line.
- Days 3-6: thin vertical slice on their data (ingest -> retrieve -> answer with citations), auth and PII rules first.
- Days 7-10: eval on the real asks with a control, fix the top failure classes (wording, retrieval, context), add guardrails and cost caps.
- Days 11-14: pilot with a few users, telemetry per turn, feedback buttons, a written readout: what works, what doesn't, what it would take.

4. AI Product Manager

Q28. How do you decide what an AI feature should do? [Intermediate]

- Start from the job: teachers want ready-to-teach material from *their* curriculum, at their students' level, in fewer clicks.
- Write the ask list from real usage (thumbs feedback, QA rounds, support tickets), then decide which asks Scout answers, which it does (with approval), and which it declines.
- CSPO practice: one backlog, value vs effort vs risk, acceptance criteria written as pass lines.

Q29. Which metrics would you track for an AI assistant? [Intermediate]

- Outcome: tasks completed (lesson plan saved, assignment created), repeat weekly use, thumbs-up rate.
- Quality: "works well" rate on a fixed eval set, failed writes, false refusals, hallucination checks.
- Experience: time to first visible text (p50/p95), turn completion rate (991/991 in the v2 baseline).
- Cost and safety: $ per turn, cap hits, guardrail blocks (true vs false), PII incidents (target 0).

Q30. Tell me about a feature you decided NOT to ship. [Senior]

- Leveller "movement arm": the idea was to skip rewriting articles already at the target level. Measurement showed the shipped leveller already rewrites nearly every article (returned >=90% similar in 0/35 and 1/28 cases), McNemar p = 1.0. It bought nothing measurable -> closed.
- Also: a grade step isn't a fixed reading distance (Flesch-Kincaid per step 1.83 at 1->2 but 0.19 at 4->5), so we don't promise "two grade levels easier" in the UI.

Q31. How do you handle AI trust, safety and governance for schools? [Senior]

- Teacher stays in control: proposals need approval; writes are logged with undo; AI-written content is labelled "Suggested by AI, can make mistakes".
- Student privacy by design (pseudonyms, allowlists), K-12 guardrails, feature flags per district.
- Gap I'd flag as PM: the governance answers (which model, does it train on our data, can a district turn it off) needed a published statement and a VPAT for accessibility - competitors are compared on these.

Q32. How do you prioritise between quality, speed and cost? [Senior]

- Set explicit bars and trade within them. Example: Haiku 5.5 improved quality but missed the 1.2x latency bar, so we held it.
- Keep a decision-ready trade on file ("~0.9 s first-visible time vs 23 of 7,104 replies changed") so leadership can unpark it with numbers.

Q33. How would you build the roadmap for Scout's next year? [Lead / Staff]

- Themes: (1) speed - reach < 2 s first visible text; (2) depth - misconception analysis and reteach loops, standards mastery; (3) admins - Scout for district leaders answering from computed figures, citing the source card; (4) trust - governance page, VPAT, quality loop inside AI reports.
- Each theme gets a metric, a gate and a kill criterion.

Q34. How do you run discovery for an AI product? [Lead / Staff]

- Interviews plus usage logs; prototype fast with real data; test with a handful of users; measure.
- At Workroom.build I ran product discovery for AI web apps and led a multinational team to a PoC (LangChain, FastAPI, Next.js, OpenAI and Gemini on GCP).

5. System design (senior / lead)

Q35. Design an AI assistant for a SaaS product with thousands of users. [Senior]

- Client: chat UI, page context published by components, action executor with verification.
- Edge/auth: reuse the product's session token; AI service validates against the app; per-tenant scoping from the server.
- Agent service: streaming endpoint; loop with typed tools; retrieval service; thread store; usage/cost store.
- Safety: guardrails (input scoped, output always), PII pseudonymisation, approvals for writes, audit log with undo.
- Ops: per-turn telemetry (turn id, stage timings, tokens, cost), budget alarms, throttles, spend caps, feature flags, eval gates in CI.
- Draw the Scout diagram: browser -> Function URL (stream) / API Gateway -> Lambda -> Bedrock + Pinecone + Mongo + DynamoDB; callbacks to Laravel.

Q36. Design the content ingestion pipeline for RAG. [Senior]

- Source of truth exports (Laravel `ai:export-content`) -> S3 with a `latest.json` pointer -> scheduled sync (ECS Fargate) -> chunk, embed, upsert with metadata (grade, language, publication, unit, week).
- Idempotent upserts, deletes as prunes with a size guard and alarm, re-index on embedding model change (dimension is fixed per index), freshness monitoring.

Q37. How would you scale to 10x traffic? [Lead / Staff]

- Bedrock quotas and cross-region inference; prompt caching; keep per-turn context small; provisioned concurrency for cold starts; DynamoDB on-demand for counters; Pinecone serverless; cache deterministic tool results; queue long jobs (grading, exports).

6. JD fundamentals: LLMs and model internals

Q38. How does a transformer work, briefly? [Intermediate]

- Text is split into tokens (BPE / SentencePiece), mapped to embeddings plus position information.
- Each layer runs self-attention - softmax(QK^T / sqrt(d)) V - so every token weighs every other token, then a feed-forward block; residual connections and layer norm around both.
- Decoder-only LLMs (GPT, Claude, Llama) predict the next token, one at a time, attending only to earlier tokens.
- Why it matters to you: cost and latency scale with tokens; long contexts cost more and quality can drop for facts buried in the middle ("lost in the middle").

Q39. Explain temperature, top-p, top-k and max tokens. [Intermediate]

- Temperature scales the logits: low (0-0.3) = focused and repeatable, high = varied/creative.
- Top-k keeps the k most likely tokens; top-p (nucleus) keeps the smallest set whose probability sums to p.
- Max tokens caps output length (and cost). Stop sequences end generation early.
- Rule of thumb: extraction/classification low temperature; brainstorming higher. Change temperature OR top-p, not both.

Q40. What is a context window and how do you manage it? [Intermediate]

- The total tokens (input + output) a model can attend to in one call.
- Manage it: summarise old turns, keep only recent messages, retrieve instead of stuffing, put the most important instructions at the start and the end, cache stable prefixes.
- **Scout tie-in:** context trimming made Nova Micro reliable in multi-turn chat; in v2 we "publish answers, not rows" and drop a whole tier with a note instead of truncating silently.

Q41. Prompting vs RAG vs fine-tuning - when do you use each? [Senior]

- Prompting first: cheapest, fastest to iterate; few-shot examples and clear output formats solve most behaviour problems.
- RAG when the model needs knowledge that is private, large or changes often, and you need citations.
- Fine-tuning when you need a consistent style/format/skill the prompt can't reliably produce, to shrink a big model's job into a small cheap model, or to cut prompt length at high volume. It does not reliably teach new facts.
- Often combined: fine-tuned small model + RAG for facts.
- **Scout tie-in:** curriculum changes hourly, so RAG; behaviour fixed by tool wording and instructions, not fine-tuning.

Q42. Explain LoRA / QLoRA and other parameter-efficient fine-tuning. [Senior]

- LoRA freezes the base weights W and learns a low-rank update BA (rank r, e.g. 8-64) on chosen layers (usually attention projections): W' = W + BA. Trains <1% of parameters, adapters are small and swappable.
- QLoRA: base model loaded in 4-bit (NF4) quantisation, LoRA adapters trained in higher precision - fine-tune a large model on one GPU.
- Workflow: curate 500-10k high-quality examples, hold out an eval set, train (Hugging Face PEFT/TRL, or Bedrock/OpenAI managed fine-tuning), compare against the prompted baseline.
- Risks: overfitting, catastrophic forgetting, data leakage of PII in training data.

Q43. What are RLHF and DPO? [Senior]

- RLHF: collect human preference pairs, train a reward model, optimise the LLM with RL (PPO) against it with a KL penalty to stay close to the base model.
- DPO (Direct Preference Optimization): skips the reward model and RL - optimises directly on chosen vs rejected pairs. Simpler and more stable, widely used now.
- Constitutional AI / RLAIF: AI feedback against written principles replaces some human labels.

Q44. Quantization, distillation and model serving - what do you know? [Senior]

- Quantization: store weights in int8/int4 (GPTQ, AWQ, GGUF for llama.cpp) - less memory, faster, small quality loss.
- Distillation: train a small student model on a big teacher model's outputs.
- Serving open models: vLLM (PagedAttention, continuous batching), TGI, Ollama locally; KV cache makes later tokens cheaper; batching raises throughput but can raise latency.
- Managed alternative: Bedrock / Azure OpenAI / Vertex - no GPUs to run, pay per token, data stays in your cloud account.

Q45. What are embeddings? [Intermediate]

- Dense vectors where similar meaning lands close together; compared with cosine similarity or dot product.
- Used for semantic search, clustering, dedup, classification, recommendations.
- Choosing a model: retrieval benchmarks (MTEB), language coverage, dimension (storage cost), max input length, price; changing the model means re-embedding everything (index dimension is fixed).

7. JD fundamentals: RAG and retrieval in depth

Q46. How do you chunk documents? [Intermediate]

- Fixed size (e.g. 300-800 tokens) with 10-20% overlap is the baseline; better: split on structure (headings, paragraphs, tables), keep metadata (title, section, page, date, permissions) on every chunk.
- Advanced: parent-child (retrieve small chunks, return the parent section), semantic chunking, contextual headers (prepend document title/summary to each chunk).
- Evaluate chunking by retrieval recall on real questions, not by intuition.

Q47. What is hybrid search and re-ranking? [Senior]

- Dense vectors miss exact terms (codes, names, SKUs); keyword search (BM25) misses paraphrases. Hybrid runs both and fuses results, e.g. Reciprocal Rank Fusion: score = sum of 1 / (60 + rank).
- Re-ranking: retrieve top 50-100 cheaply, then a cross-encoder (Cohere Rerank, bge-reranker, Bedrock rerank) scores query-document pairs and keeps the top 5-10. Big precision gain for small latency.
- Also: query rewriting, multi-query, HyDE (embed a hypothetical answer), metadata filters before vector search.

Q48. How do you evaluate a RAG system? [Senior]

- Separate retrieval from generation.
- Retrieval: recall@k, precision@k, MRR / nDCG on a labelled question -> relevant-doc set.
- Generation: faithfulness/groundedness (every claim supported by context), answer relevance, citation accuracy, refusal when context is missing. Frameworks: RAGAS, TruLens, DeepEval, promptfoo.
- LLM-as-judge with a rubric, calibrated against human labels; track judge noise.
- **Scout tie-in:** declared gates, control arms, and honesty rules ("not linked" vs "could not check").

Q49. How do vector databases work? Which would you choose? [Intermediate]

- Approximate nearest neighbour indexes (HNSW graphs, IVF, product quantisation) trade a little recall for big speed.
- Options: Pinecone (managed, serverless), pgvector (keep it in Postgres, good up to millions), OpenSearch/Elasticsearch (hybrid built in), Weaviate, Qdrant, Milvus, FAISS (library, in-process), Chroma (prototyping).
- Choose on: scale, filtering needs, hybrid support, ops burden, multi-tenancy, cost, data residency.

Q50. What are GraphRAG and agentic RAG? [Senior]

- GraphRAG: build a knowledge graph of entities and relations (often LLM-extracted), retrieve by graph traversal or community summaries - good for "how are X and Y connected" and whole-corpus questions.
- Agentic RAG: the model decides when and what to search, can search again, and checks its own sources (a tool-calling loop).
- **Scout tie-in:** `search_curriculum` is a tool the agent calls, so Scout is agentic RAG.

Q51. How do you handle permissions in RAG (multi-tenant)? [Senior]

- Store tenant/user/role metadata on every chunk and filter at query time on the server from the authenticated identity - never from what the client sends.
- Separate namespaces/indexes per tenant when isolation must be strict.
- **Scout tie-in:** publication scope comes from Laravel's auth answer, not the browser.

8. JD fundamentals: agents, tools and LLMOps

Q52. What is an AI agent? Explain ReAct and function calling. [Intermediate]

- An agent is an LLM in a loop that decides which tool to call, observes the result and continues until done.
- ReAct = reason, act, observe, repeat. Function/tool calling: you give JSON-schema tool definitions, the model returns a structured call, your code executes it and returns the result.
- Production needs: max steps, timeouts, idempotent tools, approvals for writes, logging of every step.

Q53. Compare LangChain, LangGraph, CrewAI, AutoGen, LlamaIndex and plain SDKs. [Senior]

- LangChain: building blocks and integrations. LangGraph: explicit state graph with checkpoints, good for branching/human-in-the-loop flows. LlamaIndex: data/RAG focus. CrewAI: role-based multi-agent teams. AutoGen: conversational multi-agent. OpenAI Agents SDK / Claude Agent SDK / Bedrock Agents: vendor agent runtimes.
- Plain SDK loop: least magic, easiest to debug and optimise.
- My view: prototype with a framework, keep the production core thin. (Used LangChain at Workroom, LangGraph in Scout v1, CrewAI in study.)

Q54. What is MCP (Model Context Protocol)? [Senior]

- An open protocol (introduced by Anthropic) that standardises how AI apps connect to tools and data: an MCP server exposes tools, resources and prompts; an MCP client (Claude, IDEs, agents) discovers and calls them over JSON-RPC (stdio or HTTP).
- Benefit: write an integration once, use it from any MCP client. Risks: tool poisoning, over-broad permissions - treat third-party servers as untrusted.
- Related: A2A (agent-to-agent) protocols for agents talking to each other.

Q55. When would you use multiple agents? [Senior]

- When tasks need different tools/prompts/permissions, or can run in parallel (research fan-out). Patterns: supervisor/orchestrator with workers, handoffs, planner-executor, critic/reviewer.
- Costs: more tokens, more latency, harder debugging, error compounding. Start with one agent and good tools; split only when measured.

Q56. How do you get reliable structured output? [Senior]

- Native structured outputs / JSON schema mode, or force a tool call whose input schema is the output.
- Validate with Pydantic/Zod, retry once with the validation error in the prompt, then fail clearly.
- Keep schemas small; enums for closed sets; avoid asking for JSON and prose in the same field.

Q57. What is LLMOps? What do you monitor in production? [Senior]

- Versioned prompts and models, eval sets in CI, staged rollout (shadow, canary, A/B), rollback.
- Monitor per request: latency (TTFT, total), tokens and cost, errors/throttles, tool failures, guardrail hits, user feedback, drift in input topics.
- Tools: LangSmith, Langfuse, Arize Phoenix, Helicone, OpenTelemetry GenAI conventions, CloudWatch/Datadog.
- **Scout tie-in:** per-turn `turn_id`, stage timings, `tokenz` cost, alarms and budget.

Q58. How do you reduce LLM cost and latency? [Senior]

- Smaller/faster model for easy steps (routing/cascade), prompt caching, semantic caching of repeated answers, shorter prompts and outputs, batch APIs for offline jobs (~50% cheaper), streaming for perceived speed, parallel tool calls, provisioned throughput at high volume.
- Estimate cost: requests x (input tokens x input price + output tokens x output price). Output tokens usually cost several times input tokens.

Q59. How do you A/B test an AI feature online? [Senior]

- Randomise by user (not request), pick one primary metric (task completion, retention) plus guardrail metrics (cost, latency, complaints), compute sample size before starting, run full weeks, avoid peeking.
- Offline evals decide what is safe to test; online tests decide what users value.

9. JD fundamentals: security, safety and responsible AI

Q60. What is prompt injection and how do you defend against it? [Intermediate]

- Direct: the user tells the model to ignore its rules. Indirect: instructions hidden in retrieved documents, web pages, emails or tool output.
- Defences (layered, none perfect): treat all retrieved/tool content as data and mark it as such; least-privilege tools; human approval for writes and sends; output filtering; allowlisted actions; classifiers (Bedrock Guardrails prompt attack, Llama Guard, Prompt Shields); never put secrets in prompts.
- **Scout tie-in:** the model only proposes; the browser executes allowlisted capabilities and verifies; a grader-aimed instruction inside a student answer was a real test case.

Q61. Name items from the OWASP Top 10 for LLM applications. [Senior]

- Prompt injection; sensitive information disclosure; supply chain; data and model poisoning; improper output handling; excessive agency; system prompt leakage; vector and embedding weaknesses; misinformation; unbounded consumption (cost/DoS).
- Map each to a control you have built: approvals (agency), spend caps (consumption), pseudonyms (disclosure), output checks (output handling).

Q62. Which regulations matter for AI products? [Senior]

- Education (US): FERPA (student records), COPPA (children under 13), state student-privacy laws; accessibility (WCAG, VPAT).
- General: GDPR (lawful basis, data minimisation, right to erasure), CCPA, SOC 2 for enterprise buyers, HIPAA for health.
- EU AI Act: risk tiers; education and employment uses can be high-risk (documentation, human oversight, risk management); transparency duties for chatbots and generated content.
- Vendor questions customers ask: does the provider train on our data, where is data stored, retention, can we turn it off.

Q63. How do you handle bias and fairness? [Senior]

- Define who could be harmed, build eval slices (language, grade, demographics where lawful), compare error rates across slices, keep humans in the loop for consequential decisions, document limits.
- **Scout tie-in:** AI-proposed grades are proposals a teacher confirms; feedback must not praise or judge character.

Q64. What are hallucinations and how do you measure them? [Intermediate]

- Fluent but unsupported or false output. Measure with groundedness checks against sources, citation verification, fact-check sets with known answers, and a rate on a fixed eval set.
- Reduce with retrieval, explicit "say you don't know", lower temperature for facts, structured tool results, and verification steps.

10. JD fundamentals: forward deployed and product management

Q65. How do you scope a customer AI engagement? [Intermediate]

- Business goal and the metric it moves; users and workflow today; data sources, quality and access; security/compliance constraints; integration points; what "done" means for the pilot; who signs off.
- Deliver a one-page scope with risks and a demo date.

Q66. The customer can't send data outside their cloud. How do you deploy? [Senior]

- Use the model service inside their account/region (Bedrock, Azure OpenAI, Vertex) with private networking (VPC endpoints/PrivateLink), or self-host an open model (vLLM on their GPUs / Kubernetes).
- Infrastructure as code (Terraform), container images, secrets in their vault, logs kept in their account, no training on their data.

Q67. Which engineering skills does an FDE need beyond AI? [Senior]

- Python and TypeScript, SQL, REST/GraphQL, auth (OAuth/SSO/SAML/JWT), Docker, Kubernetes basics, Terraform, CI/CD, cloud networking, data pipelines/ETL, debugging in someone else's codebase, and clear writing for customers.
- Your proof: 10+ years full stack, Laravel/Node/NestJS/FastAPI, AWS and GCP, Terraform, SSO-integrated platform.

Q68. Demo went badly in front of the customer. What do you do? [Senior]

- Stay calm, explain what happened in plain words, show the fallback (recorded run or a different case), take the failing input home, come back with root cause and fix within a day.
- Prevent: rehearse on their real data, pin model versions, have a smoke test before every demo.

Q69. How do you write a PRD for an AI feature? [Intermediate]

- Problem and users; jobs to be done; what the AI does / does not do; inputs, outputs and UX for failure (low confidence, refusal, error); eval set and pass bars; safety, privacy and compliance; cost budget per use; rollout and kill switch; success metrics.

Q70. Build vs buy for AI capabilities? [Senior]

- Buy (API/vendor) when it's not your differentiator and speed matters; build when the workflow, data or UX is your moat, or when cost/control at scale demands it.
- Consider: data privacy, vendor lock-in (abstract the model layer), total cost of ownership, team skills.

Q71. How would you price an AI feature? [Senior]

- Know unit cost (cost per task) first. Options: included in a seat (drives adoption, cap usage), usage/credits, premium tier, outcome-based.
- Protect margin with caps, model routing and caching.
- **Scout tie-in:** credits existed but became a kill switch; spend caps protect cost per teacher.

Q72. How do you define a north-star metric for an AI assistant? [Lead / Staff]

- Something that captures value delivered, e.g. "weekly teachers who completed a task with Scout (saved a plan, assigned, graded)", backed by input metrics (activation, success rate, latency, cost per task) and counter-metrics (false refusals, harmful outputs, undo rate).

Q73. How do you manage stakeholder expectations about AI accuracy? [Lead / Staff]

- Share eval results with examples, explain error types and what the human still owns, set a target and a date, label AI output, give users undo and feedback paths, report progress on the same eval set every release.

11. Issues we faced building Scout - and how we tackled them

Use these for "hardest problem", "production incident", "trade-off" and "what did you learn" questions. Pick 3-4 and know them deeply.

#	Issue	How it was tackled
1	Buffered responses felt slow; API Gateway can't stream and cuts off at 29 s	Moved the turn to a streaming Lambda Function URL with Bedrock ConverseStream and typed SSE events; one container image serves both doors; streaming rollback by alias.
2	LangGraph/LangChain added layers and state that were hard to debug	Replaced with a plain tool loop; copied v1 threads into `core_legacy_threads` with an apply/verify CLI; integration run showed no regression on any production-passing chain.
3	Old chats opened blank after removing LangGraph	Migration step per environment before the release, rerun after, then `--verify`; kept checkpoints until verify clean in prod; documented the ordering.
4	Chip-started threads returned 409 when the teacher typed a follow-up	Unified on one thread store with one title per thread; browser retries a busy/stale thread with bounded backoff.
5	Guardrail blocked normal history curriculum (wars, slavery) and showed an error card	Input check reads only the teacher's typed text; curriculum profile for curriculum asks; refusal shown as a sentence, not an error. False blocks 6 -> 0 of 240 turns.
6	PII ANONYMIZE was treated as BLOCKED and failed the whole turn	Read the guardrail trace; only a policy that actually fired BLOCKED stops the turn.
7	Newer fast model mislabelled curriculum requests in the guardrail router	Measured 105/279 read as OTHER; pinned the classifier to Haiku 4.5 regardless of the fast model.
8	Student names could reach the model	Pseudonyms from a stable alias module, real names restored at render; allowlist for facts; server-side scrub; titles never name a person; fail closed without the key.
9	DOM scraping for page context was brittle and leaky	Components publish subject/view/content/capabilities via `useAiSurface`; surfaces stack by mount depth so dialogs shadow the page.
10	Scout didn't know a modal was open / acted on the wrong layer	Dialogs publish their own surface; z-order rule: whatever opened last is on top.
11	Actions reported success that hadn't happened	Executor ignores a resolved `run()` and re-reads app state in `verify()`; server actions logged with undo; confirm re-checks for drift and saves the new preview.
12	Model invented ids (passed "1" from "Article 1: ..." as an article id)	Ids only from tool results/page facts, validated; tool errors say what would be right.
13	Fabricated counts ("I found 10 old classes" for an account with none)	Page publishes explicit zero facts; tools return answers not rows; honesty rules in retrieval ("not linked" vs "could not check").
14	Approval card promised "Remove all" when the action removes one	Card text derived from the actual capability arguments; pinned with a regression check.
15	Prompt cache re-paid on every page switch; cache writes were 63% of cost	Moved page-varying content after the cached prefix; fixed tool order; measured cache read share.
16	First visible text p50 3.18 s vs < 2 s goal	Stage timing, cold-start separation (7.7%), fast model with thinking off, early status events, smaller per-turn context. Honest status: not yet met.
17	Cost control and abuse	Daily/weekly spend caps, per-user throttles in their own buckets, in-flight lease (429), AWS budget and alarms; credits kept only as a kill switch.
18	Deploys went green with broken config; `/health` healthy with no credentials	Readiness audit; read startup logs; smoke turns after deploy; Parameter Store vs .env precedence documented.
19	Secrets exposed in the working tree / shared prod credentials	Rotated secrets, separate prod credentials, MongoDB IAM auth, security scan fixes through review.
20	Index could be wiped by a bad export	Large prunes refused unless forced, with a CloudWatch alarm; hourly sync replaced a clock-based schedule.
21	Weak answers in Spanish and early grades	Measured: 27% of Spanish weeks thin, early-grade text lived in images; handed the content team a concrete list instead of prompt-patching.

#	Issue	How it was tackled
22	Tests that couldn't fail; fixtures that lied; local data that was junk	Mutation-check new pins; gates declared before rows; control arms; measure only on real data (local graded answers were seeded junk).
23	6,350-line chat component that was hard to change	Split into sidebar, composer, message list, footer, toolbar and composables (ONLINE-8027).
24	AI changes leaking into non-AI pages	Feature flags as parent/child, page awareness runs only for users with Scout on, non-AI impact audits and parity tests before merge.
25	Second browser tab sending while a turn was running	Service refuses with a sentence; browser shows Scout's "still answering" message, not a failure card; Stop sends a cancel by turn id.
26	Knowledge lost when notes folder disappeared	Rebuilt 375 research notes from transcripts, curated an index and a decisions log in the repo; onboarding explorer and deck for new developers.

Lessons to say out loud

- When the assistant does something wrong, the fix is almost never a bigger model: it's tool result wording, instruction placement, an enum that couldn't express the ask, or an error that says "wrong" without saying what's right.
- Declare the pass line before collecting rows, and always run a control.
- Publish answers, not rows: the cost of a feature is what it adds to every turn.
- Never truncate silently: drop a whole tier of data and say so.
- The model proposes; the app executes and verifies.

12. Stories from earlier roles

Workroom.build - Lead Developer (2024)

- Multi-model agent (LangChain, FastAPI, Next.js, OpenAI + Gemini 1.5 Flash/Pro) that enriches lists, researches companies and processes hundreds of documents through a table UI; deployed on GCP.
- Use for: multi-model routing, cost vs quality (Flash vs Pro), product discovery, leading a multinational team.

Groupshop - Tech Lead (2021-2024)

- Shopify upsell app: NestJS, TypeORM, GraphQL, MongoDB; AWS Elastic Beanstalk + Amplify with CI/CD; TDD cut production errors ~15%.
- Use for: owning delivery under deadlines, architecture, mentoring.

ENN Technologies / Auto360 / InGenious (2016-2021)

- React component library with Storybook; query optimisation (+10%); mobile architecture (+20% performance); Laravel reusable libraries (-25% delivery time).
- Use for: full-stack depth - an AI engineer still has to ship the UI, API and database.

13. Questions to ask the interviewer

- How do you evaluate model changes before they reach users today?
- Who owns the decision when quality, latency and cost disagree?
- What does a forward deployed engineer's first 90 days look like here?
- How close do engineers work with customers, and how does that feedback reach the roadmap?
- What is the hardest AI failure you've had in production?

14. Before you walk in

- Your resume says LangGraph, Nova Micro, DynamoDB schema and "client plugin that collects page context from the DOM". Those describe Scout v1 and the migration plan. Today Scout runs a custom loop on Claude via Bedrock, MongoDB threads (DynamoDB for caps/throttles) and published page surfaces. Tell it as an evolution - "we started with X, measured, moved to Y" - interviewers love that story. Consider updating the resume to match.
- Say "we" for team decisions and "I" for what you personally built. Have one concrete example ready for each "I".
- Quote numbers with their caveats (n=1, target not met). It reads as senior, not weak.
- Do not share internal names, hosts, ids or customer data; the numbers in this guide are fine to discuss at a high level - check your NDA.